学术学位硕士研究生学位论文

开 题 报 告

语义约束下的跨域实时任务自动合成与时间感知优化研究

学科专业： 计算机科学与技术

研究方向： 实时系统与编译优化

学　号： **【请填写】**

学生姓名： **【请填写】**

导师姓名： **【请填写】**

报告时间： 2026 年 10 月 25 日

目 录

章节	页码
1 绪论	1
2 国内外研究现状	2
2.1 实时 Linux、隔离执行与复合实时系统	2
2.2 编译器内核化与实时编译优化	2
2.3 实时 DAG 映射、任务融合与异构部署	3
2.4 跨域通信与时间语义	3
2.5 研究现状评述	4
3 课题研究工作	4
3.1 课题研究目标及意义	4
3.2 课题的研究内容	5
3.3 拟采取的研究方案	6
3.4 拟解决关键问题	8
4 课题开发基础条件	9
5 课题工作计划	9
参考文献	10

1 绪论

随着工业控制、机器人、边缘计算和信息物理系统的发展，越来越多的周期控制程序既需要 Linux 丰富的软件生态，又需要实时操作系统或隔离执行域提供稳定的时间行为。典型控制链通常由采样、滤波、控制计算和执行器提交等阶段组成。传统实现往往将整条控制链固定部署在单一操作系统中，或由工程人员手工拆分到 Linux、RTOS 与虚拟化分区。前者容易受到调度、中断和共享资源干扰，后者则高度依赖开发人员经验，难以系统判断拆分是否合法、通信代价是否可接受以及端到端期限是否仍能满足。

现有编译器优化主要围绕指令选择、循环优化、函数内联和优化序列搜索展开。对于实时程序，最坏执行时间（WCET）导向的编译优化已经形成较成熟的研究方向。然而，端到端响应时间不仅取决于单个函数的执行时间，还受到任务释放、调度、阶段间交接、跨域通信以及输入输出时间语义的共同影响。仅缩短某个函数的机器指令路径，未必能降低整个控制事务的响应时间，更不能自动回答“哪些阶段应该融合”“哪个阶段应在 Linux 或 NuttX 中执行”“跨域交接是否值得”等系统级问题。

K-Fusion 项目前期已经建立了基于 LLVM 的程序内核化工具链，能够对受支持的 C 程序执行调用重映射、定点化、死代码消除和目标代码生成，并面向 Linux 内核模块与 NuttX 任务提供双后端。前期实验进一步表明，计算执行时间与唤醒抖动是两个需要分开讨论的量：编译优化主要作用于计算路径，而较大的唤醒长尾通常来自调度器、中断、虚拟化和共享资源。由此，本课题不再把“进一步降低某一次唤醒抖动”作为唯一目标，而是研究如何由编译器自动推导程序的执行约束、生成合法的跨域执行结构，并围绕端到端响应时间和期限可行性选择部署方案。

本课题拟研究一种效果约束的跨域任务合成方法 ECoS（Effect-Constrained Cross-domain Synthesis）。给定受支持的周期 C 程序、周期与期限契约、目标平台能力描述以及可观测行为定义，系统自动提取函数效果、共享状态与任务依赖，生成满足执行上下文和语义约束的融合与部署候选，在 Linux 内核线程、NuttX 任务及跨域通信之间进行联合选择，并输出目标代码、拒绝原因、时序估计和可追溯证据。

课题的研究边界需要明确。第一，本课题不承诺支持任意 C/POSIX 程序，而是在定义清晰的周期程序子集上研究保守、可诊断的自动转换。第二，程序可转换性与硬实时期限保证是两个不同问题。若缺少可信的 WCET/WCRT 上界，本课题只报告实测期限可行性、响应时间分布和模型误差，不把有限样本的最大值写成硬实时保

证。第三，Linux、NuttX、Jailhouse 和 IVSHMEM 是研究载体，核心问题是从程序语义推导合法执行结构及时间决策，而不是简单集成多个现成组件。

2 国内外研究现状

2.1 实时 Linux、隔离执行与复合实时系统

PREEMPT_RT 通过线程化中断、可抢占内核路径和实时调度机制改善 Linux 的实时响应能力[1]。CPU 隔离、IRQ 亲和性和无滴答配置可进一步降低干扰，但系统仍可能受到设备中断、缓存竞争、后台内核活动和虚拟化开销影响。Jailhouse 等静态分区型监控器通过预先划分 CPU、内存和设备资源，使 Linux 根分区与裸机或 RTOS 非根分区并存[2]。NuttX 提供接近 POSIX 的嵌入式实时执行环境，适合作为隔离控制任务的目标系统[3]。

复合实时操作系统研究通过 Linux 与轻量实时系统协同，兼顾丰富功能和关键任务的时间确定性。cRTOS 等工作证明了在分区架构中运行富实时应用的可行性[4]。这类研究主要从操作系统结构、服务代理和分区调度角度解决问题，但通常把任务边界、部署位置和目标实现视为已给定输入。对于一份原始 C 控制程序，如何自动提取可迁移阶段、判断执行上下文、合成跨域通信并保持状态语义，仍需要编译器层面的系统研究。

K-Fusion 的前期平台实验已经观察到运行域对时间行为的显著影响，也发现必须严格统一测量口径。唤醒延迟、定时器间隔偏差、函数执行时间和端到端响应时间不能直接相互替代。不同板卡、不同实验窗口和不同探针得到的极值也不能直接计算“性能倍数”。这些经验促使本课题把统一的 release、start、finish 时间戳和 release_id 作为后续实验的基础。

2.2 编译器内核化与实时编译优化

编译器内核化可将用户态程序中的受支持逻辑转换为内核或受限运行时可执行形式。其难点不仅在生成目标代码，还在于系统调用、动态内存、同步、设备访问和错误路径的语义转换。若对未知外部调用采用静默删除、固定返回成功或“黑洞”替换，虽然产物可能通过链接，却无法保证业务语义。因此，安全的自动内核化应当对未建模行为采取 fail-closed 策略，并给出可定位的诊断。

实时编译优化通常围绕 WCET 分析与优化序列选择展开。Lavinium 研究了现代 LLVM 上面向 WCET 的函数级编译自动调优[5]，说明优化序列搜索本身已经不是空白。K-Fusion 前期也具有函数级 Pass 调度和搜索框架，但早期代理目标主要由热路

径 IR 指令数和目标文件大小组成，不能直接代表 WCET、端到端响应时间或期限违约率。因此，本课题不把“再增加一种遗传算法或贝叶斯优化”作为主要创新，而把代码变体选择放入更高层的任务融合与执行域联合决策中。

Alive2 等翻译验证工具可以检查部分 LLVM IR 优化的等价性[6]，但跨过程、外部调用、操作系统服务和跨域通信超出了单纯 IR 等价检查的能力范围。本课题拟采用分层证据：局部 IR 变换尽可能使用翻译验证或静态检查，跨阶段和跨操作系统行为使用契约化可观测量、差分测试、故障注入和实机 trace 验证。

2.3 实时 DAG 映射、任务融合与异构部署

实时 DAG 任务映射研究通常在任务图、节点执行代价和候选处理单元已知的前提下，优化响应时间、能耗或可调度性。已有研究对异构平台上的实时 DAG 多指标优化进行了系统建模[7]。任务融合也常用于减少调度、通信和缓存开销。然而，若任务图和候选实现都由人工预先提供，求解器并没有解决“如何从原始程序得到合法任务单元”的问题。

本课题与传统 DAG 映射的差异在于：任务图、读写依赖、上下文限制和可融合边界需要从 C/LLVM 程序及少量外部契约中自动导出；融合会改变任务图结构和跨域交接数量；部署位置与融合边界必须联合选择。例如，Sensor 阶段若访问 Linux 所有的设备，Filter 和 PID 可以在某些条件下融合到 NuttX，但 Sensor 不能被直接迁移。编译器应当自动排除非法布局，而不是让求解器在人工清洗后的候选集合上工作。

2.4 跨域通信与时间语义

共享内存、VirtIO/RPMsg 和 OpenAMP 等机制可以实现异构处理器或多执行域通信[8]。但通信框架本身并不决定任务如何划分，也不会自动保证实时期限。对于周期控制程序，物理传输耗时与依赖生效时间还可能存在数量级差异。K-Fusion 前期在 Pi4/Jailhouse/NuttX 上测得，IVSHMEM 轮询通路的单程中位数约为 0.65 μs；当实时侧只在 1 ms 周期边界消费数据时，控制依赖的中位生效时间约为 1005 μs。由此，跨域边在规划模型中应按“拍”建模，而不能仅使用空转微基准的亚微秒数值。

逻辑执行时间（LET）研究通过规定输入采样和输出提交时刻提高时序可组合性[9]。本课题借鉴其显式时间语义思想，但不会在用户未授权时把立即提交语义自动改写为 LET。契约将区分 immediate 与 LET，融合和跨域转换必须保持所声明的采样、状态更新与输出顺序。

2.5 研究现状评述

综合来看，实时操作系统与分区技术解决了“在哪里运行”的基础设施问题，WCET 编译优化解决了“如何优化单个代码实现”的问题，DAG 映射解决了“如何放置已知任务”的问题，跨域通信框架解决了“如何传消息”的问题。仍缺少一条完整链路：从原始周期 C 程序出发，自动推导执行效果和依赖，生成合法融合单元及跨域通信，并以端到端时间目标联合选择执行结构。

本课题拟填补的不是上述所有领域，而是它们之间的编译决策空隙。预期贡献能否成立，需要通过强基线和消融实验验证：若完整联合策略与简单的固定融合或启发式映射差异落在测量不确定性内，就不宣称联合优化具有显著优势；若跨域成本始终高于潜在收益，规划器选择单域方案同样是正确且有价值的结果。

3 课题研究工作

3.1 课题研究目标及意义

本课题的总体目标是设计并实现一个面向周期实时 C 程序的语义约束跨域任务合成系统，使编译器能够在明确定义的程序子集上完成“自动分析—合法性判断—执行结构合成—双后端生成—时序评估—证据输出”的闭环。

具体目标包括：

1. 构建跨过程效果与上下文分析，自动识别纯计算、状态读写、睡眠、阻塞 I/O、MMIO、动态分配、未知外部调用等行为，并以保守传播方式生成函数和阶段摘要。
2. 从调用关系、读写集合、线程入口和周期相位中恢复任务 DAG，区分同拍依赖、跨周期反馈边和无序写冲突，遇到不可解析调用时明确拒绝或标记不确定性。
3. 建立保持输入采样、状态更新和可观测输出顺序的任务融合规则，自动生成 Linux kthread 与 NuttX task 后端代码。
4. 将跨域执行域、融合边界、通信边和代码变体放入统一规划模型，以端到端期限优先、尾部风险和资源约束为目标选择候选。
5. 建立静态拒绝、语义差分、故障注入和实机时序测量相结合的验证体系，形成包含源码、计划、环境指纹、产物哈希和 trace 的证据包。

本课题的理论意义在于把执行效果分析与实时任务结构优化联系起来，研究从程序语义自动构造可行域的方法；工程意义在于降低跨 Linux/RTOS 手工移植和反复调

参的成本，并使非法转换能够在部署前被诊断；实验意义在于建立计算时间、唤醒延迟、通信成本和端到端响应时间相互区分的测量方法。

3.2 课题的研究内容

3.2.1 效果约束与执行上下文分析

基于 LLVM New Pass Manager、调用图、别名分析、MemorySSA、循环分析和目标 API 语义库，为每个函数生成效果摘要。摘要至少包含 effects、allowed_contexts、reads、writes、已知调用和未知调用。跨过程传播采用不动点计算；递归、未知函数指针或未建模外部函数不能收敛时标记为 UNKNOWN。平台能力表描述 Linux kthread、软中断、硬中断和 NuttX task 的允许行为，合法性检查默认保守拒绝。

3.2.2 任务图与状态依赖恢复

从函数调用、实参绑定、全局状态和线程创建关系构造任务图。对阶段之间的 RAW、WAW、调用和 spawn 边分别建模，防止把调用传播产生的同一依赖重复计算。周期反馈边使用 z^{-1} 标识，使代数环与合法的跨周期反馈环能够区分。对小规模人工真值图计算 precision、recall 和 F1，并设置负例验证未知依赖、假环和相位信息缺失时的 fail-closed 行为。

3.2.3 语义约束的选择性融合

对同一周期内相邻阶段生成连续分组。只有在执行上下文交集非空、状态顺序可证明、无未授权外部可观测副作用、绝对采样/提交窗口不被破坏时才允许融合。对于反馈控制环，融合还必须满足死区上限和闭合边跨组约束。规划器输出方案后，由独立验证器重新计算域纯度、阶段覆盖、组间顺序、时间窗口、积压和期限，防止生成器直接信任规划结果。

3.2.4 跨域代码合成与通信协议

对于合法跨域方案，将父计划按执行域拆分为 Linux 与 NuttX 子计划，生成各自的周期宿主和共享周期体。跨域数据依赖从 DAG 边的证据中提取，自动生成发布/订阅桥。通信采用固定大小 SPSC 环，协议包含版本、session、序号、release_id 和负载；写端先发布负载再提交序号，读端检查回绕、残留 session、过

期和丢帧。第一阶段使用已有 IVSHMEM 通路，OpenAMP/RPMsg 仅作为后期可替换通信后端和对照，不作为核心创新。

3.2.5 时间成本模型与联合规划

为阶段计算、任务释放、同域交接和跨域依赖分别建模。令任务图为 G=(V,E)，每个阶段 v 的决策变量包含执行域 d、执行上下文 c 和代码变体 o；相邻阶段的融合变量表示是否属于同一执行组。候选方案的响应时间由释放代价、阶段执行代价、同域/跨域交接及干扰项构成。跨域边根据消费相位增加整数个周期，而不是直接使用共享内存微基准。

优化采用词典序目标：首先排除语义非法和期限不可行方案，其次最小化期限违约及高分位响应时间，再考虑代码大小与搜索成本。小图使用穷举结果作为 oracle；规模增大后采用约束求解和关键路径引导的候选搜索。测量反馈只能更新经验成本模型，不能被描述为静态 WCET 证明。

3.2.6 验证、实验与可复现证据

静态层验证未映射符号、ABI 尺寸、上下文冲突、未知调用和共享状态风险；功能层使用相同输入种子及边界条件比较输出和状态序列；系统层记录计划释放 r、阶段开始 s、跨域发布/接收和最终完成 f。核心指标包括端到端响应 R=f−r、期限违约率、p50、p99、p99.9、CVaR、最坏观测值及置信区间。所有结果绑定源码、计划、编译选项、内核/固件版本和硬件指纹。

3.3 拟采取的研究方案

总体技术路线如图 1 所示。

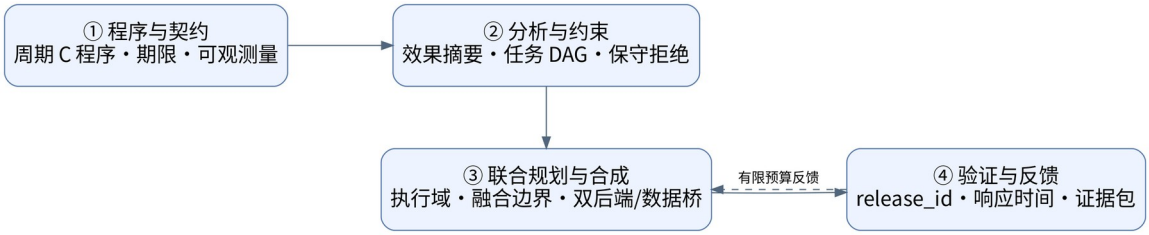


图 1 ECoS 跨域实时任务合成流程

3.3.1 输入模型与受支持子集

第一阶段限定为单个周期事务：有界循环、确定性数值计算、显式输入输出结构体、显式状态对象、可静态解析或保守枚举的调用，无未建模动态分配、阻塞 I/O 和

隐式设备所有权。用户只需给出周期入口、周期/期限、可观测量和允许执行域，不能直接指定最终融合方案。该限制使研究问题可验证，也便于量化假拒绝率和覆盖率。

最小示例采用 Sensor→FIR→PID 三阶段控制链，周期为 1000 μs，期限扫描 200、500 和 800 μs，计算利用率设置为 5%、20%、50% 和 80%。候选结构包括 A|B|C、AB|C、A|BC 和 ABC，候选域包括 Linux-only、NuttX-only 及 Linux↔NuttX。若某阶段具有平台专属 MMIO 效果，不允许的域应在候选生成前被排除。

3.3.2 ECoS 算法流程

算法分为五步：第一步从 LLVM IR 提取任务图、效果摘要和状态对象；第二步结合平台能力生成上下文、所有权、融合和时间窗口约束；第三步枚举或求解合法分区、域选择与代码变体；第四步对候选进行独立验证、代码生成和语义差分；第五步在有限预算内实机测量 Pareto 前沿候选并更新成本模型。若所有候选被拒绝，系统输出结构化拒绝码和最小冲突原因。

对小图，穷举全部合法组合并与求解器结果比对，验证规划器正确性。对较大图，使用关键路径下界、域所有权和效果冲突优先剪枝。经验预测只用于排序，不能作为安全剪枝的唯一依据。这样既控制搜索成本，又保留对决策依据的解释能力。

3.3.3 双后端与跨域生成

Linux 后端采用实时内核线程执行可睡眠业务逻辑，HRTimer 仅用于触发或打点，不把任意业务代码默认放入硬中断。NuttX 后端以任务/线程运行相同共享周期体。跨域方案先拆分为域纯子计划，再通过自动生成的 IVSHMEM 桥交换上一拍或契约指定的消息。生成器必须拒绝未验证计划、跨域混合组和缺少数据证据的跨域边。

3.3.4 实验设计

基线包括：B0 原生 Linux 用户线程；B1 现有 K-Fusion 固定 Linux 内核线程；B2 固定 NuttX task；B3 固定域全合法融合；B4 固定域不融合并做函数级 Pass 调优；B5 已给定 DAG 的异构映射算法；B6 小图全组合穷举；以及完整 ECoS。所有基线使用相同控制逻辑、算术精度、CPU 资源预算和实验窗口。

消融包括：去掉效果剪枝、固定融合边界、固定执行域、去掉时间反馈、去掉关键路径排序和固定 -O2 代码变体。评价合法候选率、错误接受/错误拒绝、编译次

数、搜索时间、端到端响应、期限违约率和模型残差。最终结果至少进行多个独立会话，并保留原始 trace 与置信区间。

3.3.5 前期工作基础与当前进度

截至开题时，项目已经完成任务契约与平台能力 schema、效果摘要分析、自动 DAG、单域融合规划、独立方案验证、Linux/NuttX 双后端生成、IVSHMEM 最小运行时、跨域成本按拍建模以及跨域计划按域拆分与数据桥生成。现有自动化回归包含效果分析 33 条、DAG 35 条、契约 8 条、融合规划 95 条、后端合成 79 条，共 250 条断言。

前期实测表明：四阶段链全融合时端到端响应均值约 22.86 μs，与模型估计 22.86 μs 一致；分组执行因相位等待可达到约 620 μs，说明减少调度释放与相位等待是融合收益的主要来源。跨域 IPC 空转单程中位数约 0.65 μs，但 1 kHz 周期消费下依赖生效中位数约 1005 μs，验证了“通路延迟不等于依赖代价”。这些结果属于前期可行性证据，不能替代后续完整的联合优化与多会话实验。

下一直接里程碑是 PR10-D：把自动拆分生成的两端产物同时部署到 Pi4/Jailhouse/NuttX，使用匹配的 release_id 对账业务状态和时间线，验证跨域桥是否存在错序、丢帧、重复消费或多拍滞后。此后再建设统一成本数据库、完整联合求解器、反馈优化和论文实验矩阵。

3.4 拟解决关键问题

（1）如何从 C/LLVM 程序中保守推导执行效果。指针别名、递归、函数指针和外部库会使效果传播不完整。拟通过受支持子集、目标 API 摘要、别名分析和 UNKNOWN 传播控制风险，并分别统计假拒绝与错误接受。

（2）如何定义融合前后的语义关系。融合可能改变输入采样、状态提交、错误传播和外部 I/O 顺序。拟以周期事务、可观测量、immediate/LET 时间语义和 release_id 为基础定义等价边界，并通过差分 trace 与局部翻译验证建立证据。

（3）如何正确建模跨域依赖成本。共享内存传输本身可能很快，但周期消费者只能在特定边界读取数据。拟把物理通路延迟、轮询粒度和依赖跨拍数分开记录，并在规划器与独立验证器中使用相同的域语义模型。

（4）如何使联合搜索既可解释又可扩展。域、融合和代码变体会产生组合爆炸。拟先用合法性约束缩小可行域，小图用穷举验证，大图使用关键路径引导与有限预算反馈，并报告最优性差距而不是无依据宣称全局最优。

（5）如何避免测量自扰和统计误用。实时尾部事件稀少，日志、磁盘 I/O 和监控线程本身也会影响结果。拟使用预分配环形缓冲离线导出，统一时钟与环境指纹，进行多会话重复、置信区间和 hold-out 验证，明确区分观测最大值与可证明上界。

4 课题开发基础条件

4.1 主要硬件设备

1. Raspberry Pi 4B：作为主要 Jailhouse/cRTOS 对照平台，运行 PREEMPT_RT Linux 根分区与 NuttX 非根分区。
2. Raspberry Pi 5：用于额外的平台泛化验证，不与 Pi4 数据直接计算性能改善倍数。
3. 串口调试器、GPIO/逻辑分析仪：用于 NuttX 启动日志、外部时基校验和必要的端到端边界测量。
4. 开发主机与存储设备：用于 LLVM 构建、固件生成、实验归档和数据分析。

4.2 主要软件平台

1. 编程语言与工具：C/C++17、Python 3、Shell、CMake、Ninja/Make。
2. 编译基础设施：LLVM/Clang 16 为板端基线，LLVM 19 用于云端兼容验证；LLVM New Pass Manager、AliasAnalysis、MemorySSA 等。
3. 运行平台：Linux PREEMPT_RT、NuttX、Jailhouse、IVSHMEM；OpenAMP/RPMsg 作为后期可选对照。
4. 验证与分析：现有自测框架、JSON Schema、差分 trace、llvm-nm/objdump、可选 Alive2/SMT 工具。
5. 项目基础：K-Fusion 已具有双后端内核化、统一探针、平台脚本、实验数据和论文图表生成工具，可支撑后续增量研究。

5 课题工作计划

工作计划以当前日期为起点，具体月份可根据学校中期检查和答辩安排调整。

时间	具体计划与验收结果
2026.10—2026.11	整理开题材料与相关工作；冻结 Pi4/LLVM16/Jailhouse/NuttX 基线；完成 PR10-D 两端联合上板、release_id 对账和故障注入。
2026.12—2027.02	完善统一 trace schema 和成本数据库；分离

	阶段计算、释放、同域交接和跨域整拍代价；建立环境指纹与重复实验流程。
2027.03—2027.05	实现执行域、融合边界和代码变体的联合约束模型；加入 3—8 阶段穷举 oracle、UNSAT 解释和独立方案复核。
2027.06—2027.08	实现有限预算反馈、候选缓存、Pareto 选择和关键路径排序；完成第二应用与高利用率、多任务扩展。
2027.09—2027.11	完成 B0—B6 基线、A0—A6 消融、多会话与不同干扰情景实验；分析模型残差、假拒绝率和适用边界。
2027.12—2028.02	整理实验结果，补充相关工作与威胁分析；形成可复现 artifact 和阶段性论文，按实验事实修订研究主张。
2028.03—2028.05	完成学位论文撰写、修改、预答辩和正式答辩材料。

本课题设置以下停止条件：若正例无法保持每周期可观测行为一致，则暂停性能比较；若静态检查对危险负例出现错误接受，则停止真实内核部署；若完整 ECoS 与强启发式的差异处于测量不确定性内，则不宣称性能优势；若无法建立可信 WCET/WCRT 上界，则全文只使用“实测期限可行性”，不使用“硬实时保证”。

参考文献

[1] The Linux Kernel Documentation. How realtime kernels differ: Timers[EB/OL]. https://docs.kernel.org/core-api/real-time/differences.html.

[2] Siemens AG. Jailhouse: Linux-based partitioning hypervisor[EB/OL]. https://github.com/siemens/jailhouse.

[3] Apache Software Foundation. Apache NuttX Documentation[EB/OL]. https://nuttx.apache.org/docs/latest/.

[4] YANG C F, et al. Compounded Real-Time Operating Systems for Rich Real-Time Applications[J]. IEEE Access, 2025.

[5] MAGNANI G, et al. Modern LLVM-based Compiler Autotuning for WCET Optimization[C]//Proceedings of the IEEE Real-Time Systems Symposium. 2025. DOI:10.1109/RTSS66672.2025.00043.

[6] LOPES N P, et al. Alive2: Bounded Translation Validation for LLVM[C]//Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. 2021: 65-79.

[7] Multi-Criteria Optimization of Real-Time DAGs on Heterogeneous Platforms under P-EDF[J]. ACM Transactions on Embedded Computing Systems, 2023. DOI:10.1145/3592609.

[8] OpenAMP Project. OpenAMP Documentation[EB/OL]. https://openamp.readthedocs.io/.

[9] GEMLAU B, et al. System-level Logical Execution Time[R]. 2021. https://inria.hal.science/hal-03125851/.

[10] LLVM Project. LLVM Language Reference Manual[EB/OL]. https://llvm.org/docs/LangRef.html.

[11] LLVM Project. LLVM Alias Analysis Infrastructure[EB/OL]. https://llvm.org/docs/AliasAnalysis.html.

[12] BUTTAZZO G C. Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications[M]. 3rd ed. New York: Springer, 2011.

[13] SHA L, RAJKUMAR R, LEHOCZKY J P. Priority inheritance protocols: An approach to real-time synchronization[J]. IEEE Transactions on Computers, 1990, 39(9): 1175-1185.

[14] DAVIS R I, BURNS A. A survey of hard real-time scheduling for multiprocessor systems[J]. ACM Computing Surveys, 2011, 43(4): 1-44.

[15] KOPETZ H, STEINER W. Real-Time Systems: Design Principles for Distributed Embedded Applications[M]. 3rd ed. Cham: Springer, 2022.

[16] Siemens AG. IVSHMEM v2 Specification[EB/OL]. https://github.com/siemens/jailhouse/blob/master/Documentation/ivshmem-v2-specification.md.

[17] K-Fusion Project. K-Fusion: An LLVM-based Kernelization and Fusion Compiler[EB/OL]. https://github.com/1147426025lu-bot/K-Fusion, 2026.

[18] K-Fusion Project. ECoS Research Design: Effect-Constrained Cross-domain Synthesis[R]. 2026.

[19] K-Fusion Project. PR10 Cross-domain IPC Minimal Runtime and Measurements[R]. 2026.

[20] K-Fusion Project. Single-domain Fusion Planning and Measurements[R]. 2026.